Image Processing Based Autonomous Fire Detection and Extinguishing Robot

Shaishab Saha*, Mrinmoy Das Pranto†, Md. Shafiul Alam‡, S M Abir Hasan§, Abrar Zawad¶
Department of Electrical and Electronic Engineering
Bangladesh University of Engineering and Technology (BUET)
Dhaka, Bangladesh
*2106066@eee.buet.ac.bd, †2106073@eee.buet.ac.bd, ‡2106074@eee.buet.ac.bd, §2106076@eee.buet.ac.bd, ¶2106077@eee.buet.ac.bd

***Abstract*—This paper presents the design, implementation, and experimental evaluation of a low-cost autonomous fire detection and suppression system built entirely from locally available components. The architecture is deliberately distributed: a laptop running a custom-trained YOLOv8 convolutional neural network serves as the central perception and planning node, while a lightweight ESP32-based mobile robot handles actuation. An overhead smartphone camera provides a global view of the arena. The robot's position and heading are continuously resolved via OpenCV ArUco marker tracking, with dynamic pixel-to-centimetre calibration eliminating errors introduced by camera height variation. If the primary YOLO detector falls below confidence thresholds, a secondary HSV morphological pipeline maintains fire localisation. A command centre computes Euclidean distance and heading error each cycle, transmitting discrete steering commands ('F', 'L', 'R', 'S') over UDP with an encoder-based closed-loop burst-movement strategy on the ESP32 preventing slip-induced overshoot. Upon arriving within 15 cm of the fire, dual LM393 infrared sensors perform an independent proximity confirmation, a servo-actuated nozzle aligns with the detected flame side, and a relay-driven pump extinguishes the source. Experimental trials achieved an 80% full-system success rate. The complete prototype was built for under BDT 5,300 (approximately USD 48), demonstrating practical viability for deployment in Bangladeshi factories and public buildings.**

***Index Terms*—YOLOv8, ArUco tracking, visual servoing, UDP control, autonomous firefighting, ESP32, encoder closed-loop, HSV detection.**

I. Introduction

Fire remains one of the most destructive hazards in industrial and residential environments. In Bangladesh specifically, fires in garment factories and crowded markets have repeatedly caused mass casualties [**?**]. A critical factor is response latency: by the time emergency services arrive, significant fire spread has already occurred. Fixed sprinkler systems respond immediately but lack spatial targeting, wasting water and potentially damaging property unrelated to the fire source.

Autonomous mobile fire-suppression robots address both limitations. They respond at the moment of detection, navigate directly to the source, and apply suppression precisely. However, existing commercial platforms cost upward of USD 500 and rely on specialised hardware unavailable in local markets. The system presented in this paper achieves the same functional objective using components sourced entirely from electronics markets in Dhaka for under USD 48.

The key design insight is a *distributed architecture*: computationally intensive tasks (deep learning inference, geometric planning) run on a laptop, while the robot executes simple, well-defined movement commands. This keeps the robot hardware minimal, reduces onboard power consumption, and allows the fire detection algorithm to be upgraded without modifying the robot at all.

Prior work in this domain has progressed from simple UV/IR proximity sensors [**?**] to colour-space thresholding and, most recently, to deep learning detectors. YOLOv8 [1], [2], trained on dedicated fire datasets [3], [4], now represents the state of the art in real-time flame localisation. The contribution of this work is the integration of YOLOv8 detection with ArUco-based robot localisation [**?**], [6] and encoder closed-loop burst movement into a single coherent, low-cost, deployable system, verified through physical experiments.

II. System Architecture

The system comprises two communicating nodes as shown conceptually in Fig. 1.

Node 1 — Laptop Command Centre: Hosts the camera interface, image processing pipeline (CLAHE pre-processing, YOLOv8 primary detector, HSV fallback), ArUco localisation, navigation geometry, and UDP transmitter.

Node 2 — ESP32 Mobile Unit: Receives UDP packets, decodes movement commands, drives four DC motors through L298N H-bridge drivers using hardware-interrupt optical encoders for closed-loop control, and executes the extinguish sequence via servo and relay.

Communication uses UDP over a shared 2.4 GHz Wi-Fi network. UDP is preferred over TCP because its lower latency is more valuable than delivery guarantees in this real-time control application; the command cooldown period (1.3 s) naturally handles any isolated packet loss.

III. Vision and Detection Pipeline

A. Image Pre-Processing

Each camera frame captured from the overhead smartphone (IP Webcam, 640×480 pixels) is pre-processed using Contrast Limited Adaptive Histogram Equalisation (CLAHE) [**?**] in the LAB colour space. CLAHE operates on the L (luminance) channel only, normalising local contrast without introducing

colour shifts. This step is critical for fire detection under varying ambient lighting — particularly when the only illumination is the candle flame itself. ArUco detection uses the raw unprocessed greyscale frame to avoid CLAHE-induced edge artefacts that can corrupt marker corner localisation.

B. Primary Fire Detection: YOLOv8

A YOLOv8n (nano) model was fine-tuned on a combined dataset comprising D-Fire [3] and Roboflow fire detection annotations [4], with additional manual annotation of candle-flame images to improve small-fire sensitivity. The model runs at a confidence threshold of 0.25. Rather than using the geometric centre of the bounding box — which oscillates vertically as the flame flickers — the system extracts the *bottom-centre* coordinate $(x_f, y_f) = \left(\frac{x_1+x_2}{2}, y_2\right)$ of the detected box. This point is stable because it corresponds to the physical fuel source at floor level regardless of flame height variation.

C. Fallback Detection: HSV Morphological Pipeline

When YOLO confidence falls below threshold (e.g., severe occlusion or extreme dim lighting), a secondary pipeline activates. The CLAHE-enhanced frame is Gaussian-blurred (11×11 kernel) then converted to HSV. Pixels within $H \in [0, 25]$, $S \in [120, 255]$, $V \in [150, 255]$ are isolated as candidate fire pixels. A 7×7 elliptical morphological close-then-open removes noise and fills gaps. The centroid of the largest remaining contour (minimum area: 80 px²) is taken as the fire location.

D. Temporal Continuity

A 15-frame memory buffer retains the last valid fire position. If both YOLO and HSV fail for up to 15 consecutive frames, the robot holds its last command and continues on course. If detection is lost beyond 15 frames, the pipeline resets and the robot stops to wait.

E. Robot Localisation via ArUco

A DICT_4X4_50 ArUco marker (ID 0) is affixed flat on top of the robot. OpenCV's `ArucoDetector` identifies the marker's four corner pixels. The midpoint of the top edge gives the front indicator $\mathbf{f} = \left(\frac{c_0+c_1}{2}\right)$; the midpoint of the bottom edge gives the rear indicator $\mathbf{b} = \left(\frac{c_3+c_2}{2}\right)$. The robot centre is $\mathbf{p}_b = \frac{\mathbf{f}+\mathbf{b}}{2}$.

The pixel-to-centimetre conversion factor is dynamically recalculated each frame from the physical marker side length:

$$PPC = \frac{\| c_1 - c_0 \|_2}{L_{marker}} \tag{1}$$

where L_{marker} is the printed marker's physical width in centimetres (measured once before deployment). This eliminates scale errors caused by camera height drift or accidental tilt.

IV. Navigation and Control

A. Geometric Error Computation

Each cycle, the command centre computes the Euclidean distance from the robot centre to the fire base:

$$d = \frac{\sqrt{(x_f - x_b)^2 + (y_f - y_b)^2}}{PPC} \tag{2}$$

The robot's absolute heading and the required bearing to the fire are both computed using `atan2`, accounting for the inverted y-axis of image coordinates:

$$\theta_{bot} = \arctan 2(-(f_y - b_y),\ f_x - b_x) \tag{3}$$

$$\theta_{target} = \arctan 2(-(y_f - y_b),\ x_f - x_b) \tag{4}$$

$$\theta_{err} = (\theta_{target} - \theta_{bot} + 180) \bmod 360 - 180 \tag{5}$$

The modular wrapping in (5) correctly maps all errors into $[-180, +180]$, preventing the sign ambiguity that occurs near ± 180.

B. Discrete Visual Servoing Control Law

Continuous PID is impractical here because the feedback loop is closed through a camera at approximately 10–15 fps over a Wi-Fi link with variable latency. Instead, a discrete proportional controller translates each error into a number of encoder *slots* — a slot being one encoder pulse, corresponding empirically to approximately 1 cm of forward travel or 1.5° of in-place rotation. The control law is:

$$n_{fwd} = \text{clamp}\left(\frac{d - d_{stop}}{c_{cm}},\ 1,\ N_{max}\right) \tag{6}$$

$$n_{turn} = \text{clamp}\left(\frac{|\theta_{err}|}{c_{deg}},\ 1,\ N_{turn}\right) \tag{7}$$

where $d_{stop} = 15$ cm is the arrival threshold, $c_{cm} = 1.0$ cm/slot, $c_{deg} = 1.5$/slot, $N_{max} = 20$ slots (far range), and $N_{turn} = 10$ slots (far) or 6 slots (within 25 cm). A command cooldown of $T_{cd} = 1.3$ s ensures that a new command is never issued while the ESP32 is still executing the previous burst (maximum burst duration: 1.5 s), preventing command stacking.

The priority hierarchy each cycle is: (1) if $d < d_{stop}$, send **S** (extinguish); (2) else if $|\theta_{err}| > 10$, send **L** or **R** with n_{turn} slots; (3) else send **F** with n_{fwd} slots.

C. ESP32 Encoder Closed-Loop Burst Movement

On receiving a movement command, the ESP32 resets both encoder counters to zero (inside a `noInterrupts()` critical section to prevent race conditions) then drives the motors. Each wheel side stops independently as its encoder count reaches the target, maintaining straight-line travel even if the two motors have slightly different no-load speeds:

```
while (L_count < target || R_count < target) {
  if (L_count >= target) stop_left();
  if (R_count >= target) stop_right();
  if (no_tick_for > 400ms) break; // stall
  if (elapsed > 1500ms)   break; // timeout
```

```
  delay(1); // yield to FreeRTOS WiFi task
}
stopMotors();
```

The `delay(1)` yield is critical: the ESP32 WiFi stack runs as a FreeRTOS background task and is starved by tight polling loops, causing incoming UDP packets to be silently dropped. A 1 ms yield per iteration is imperceptible to movement but restores full WiFi reception during bursts.

A 400 ms stall detector terminates the burst if neither encoder ticks (wheel blocked by obstacle), and a 500 ms watchdog in the main loop stops all motors if no packet arrives — preventing runaway behaviour on WiFi loss.

D. Smart Flush for Stop-Command Integrity

After completing any movement burst, the firmware calls `smartFlush()`, which drains the UDP buffer but executes any buffered **S** command rather than discarding it. Standard buffer draining silently discards stop commands that arrived during a burst, causing the robot to overshoot the fire target. `smartFlush()` guarantees that a stop signal sent during any burst is always honoured.

V. Hardware Implementation

A. Power Architecture

An 11.1 V, 2200 mAh 3S Li-Po battery feeds two parallel branches through a main toggle switch. The first branch (11.1 V) supplies both L298N motor drivers directly. The second passes through a LM2596 buck converter, adjusted to precisely 5.0 V, which powers the ESP32 VIN pin, the MG996R servo, and the relay module VCC. The ESP32's regulated 3.3 V output pin powers all LM393 modules (two flame sensors, two wheel encoders), keeping them within safe input voltage limits. The pump's high-current path (approximately 200–500 mA at 5 V) passes through the relay contacts, never through any GPIO pin.

B. Motor Drive and Sensing

Four TT gear motors (6 V / 200 RPM) are arranged in a 4WD configuration. Driver A controls the left pair; Driver B controls the right pair. PWM is generated at 5 kHz, 8-bit resolution using ESP32's `ledcAttach`/`ledcWrite` API. A nominal duty cycle of 190/255 (≈74%) provides sufficient torque to prevent stall on smooth flooring while remaining within encoder counting accuracy. Table I summarises the GPIO assignment.

C. Extinguish Mechanism

Upon receiving command **S**, the firmware reads the two LM393 flame sensor outputs (active LOW on detection). The MG996R servo angle is set to 45° (left-side fire), 135° (right-side fire), or 90° (centre / both). After 600 ms settling time, the active-HIGH relay closes for 3,000 ms, driving the submersible pump at 5 V through the NO contact. The servo then returns to 90°. The two-stage confirmation — overhead camera proximity check *and* onboard IR sensor verification — prevents false activations from non-fire heat sources.

TABLE I
ESP32 GPIO Pin Assignment

Pin	Device	Function	Dir.
D4	Relay Module	Pump ON/OFF	Out
D5	MG996R Servo	Nozzle Aim (PWM)	Out
D13	Driver B IN4	R-Rear Backward	Out
D14	Driver B IN1	R-Front Forward	Out
D18	Driver B IN2	R-Front Backward	Out
D19	Left Encoder	Speed Feedback	In (IRQ)
D21	Right Encoder	Speed Feedback	In (IRQ)
D25	Driver B IN3	R-Rear Forward	Out
D26	Driver A IN1	L-Front Forward	Out
D27	Driver A IN2	L-Front Backward	Out
D32	Driver A IN3	L-Rear Forward	Out
D33	Driver A IN4	L-Rear Backward	Out
D34	Left Flame Sensor	Fire Confirm	In
D35	Right Flame Sensor	Fire Confirm	In
VIN	Buck OUT+	Main 5 V Input	Power
3V3	All LM393 VCC	Sensor 3.3 V	Power

VI. Experimental Results

A. Fire Detection Performance

The YOLOv8 model correctly detected a lit candle at all five arena positions tested under standard indoor lighting. Under severely reduced lighting (candle-only illumination), the model retained detection in three of five cases; the HSV fallback took over in the remaining two, correctly estimating the fire centroid to within ±8 pixels. The bottom-centre tracking strategy reduced vertical centroid jitter from approximately ±15 pixels (centre tracking) to ±2 pixels, directly improving distance estimation stability.

B. Full-System Trials

Five complete autonomous runs were conducted from randomised starting positions. Four runs completed the detect–navigate–extinguish sequence successfully. The single failure was caused by a Wi-Fi packet loss during a 90° turn, which triggered the watchdog and halted the robot before it completed alignment. This is a networking issue rather than an algorithmic one; using a dedicated local hotspot or increasing the watchdog timeout modestly is expected to eliminate it. The overall success rate was 80%.

VII. Cost Analysis

Table II summarises the bill of materials for the prototype.

At 500-unit production scale, bulk purchasing and a custom PCB are estimated to reduce unit cost to BDT 2,200–2,500 (≈ USD 20–23), well below comparable commercial platforms (typically USD 500+).

VIII. Design Considerations

A. Safety

All high-current paths (motors, pump) are isolated from ESP32 GPIO pins by the L298N drivers and relay. The 500 ms watchdog ensures the robot stops within half a second of any WiFi disruption. The 15 cm stopping threshold keeps the

TABLE II
BILL OF MATERIALS

Component	Spec	Qty	BDT
ESP32 DevKit V1	—	1	450
L298N Motor Driver	Dual H-Bridge	2	360
TT Gear Motor + Wheel	6 V, 200 RPM	4	480
MG996R Servo	Metal gear	1	350
LM393 Flame Sensor	IR module	2	160
LM393 Encoder Module	Infrared Speed	2	180
Encoder Slit Disc	Rotary	2	70
Relay Module	5 V, 1-channel	1	60
Mini Water Pump	3–5 V DC	1	150
Buck Converter	LM2596 adj.	1	120
Li-Po Battery	11.1 V 2200 mAh	1	800
Power Switch	Toggle	1	30
4WD Acrylic Chassis	—	1	400
Wires, connectors	Assorted	—	150
Tubing + reservoir	5 mm silicone	—	100
ArUco marker (print)	A4 card	1	10
Misc. Consumables	Fare, glue, solder, breadboard	—	1,400
Total			**5,270**

robot away from direct contact with the fire source. The Li-Po battery is handled with a balance charger and stored in a fire-resistant bag.

B. Environmental Impact

Water is used as the suppression medium, causing no chemical pollution. Morphological filtering and dual confirmation logic minimise false activations, reducing unnecessary water discharge compared to undifferentiated sprinkler systems. Electronic components are designed for recyclability; the Li-Po battery is disposed of through certified e-waste channels.

C. Societal Relevance

Fire incidents in Bangladeshi garment factories and densely packed commercial spaces have caused repeated mass-casualty events. A system that responds autonomously at the moment of detection — before emergency services arrive — could significantly reduce casualties. Because all components are available in Dhaka's Elephant Road electronics market, the design is replicable by local engineers and small businesses without specialist procurement.

IX. Limitations and Future Work

The primary limitation identified is Wi-Fi packet loss on shared 2.4 GHz networks during large-angle turns, causing premature watchdog termination. Mitigation includes dedicated 5 GHz hotspot operation or a modest watchdog extension. Angular accuracy degrades for heading corrections exceeding 60° due to cumulative encoder uncertainty; integrating an MPU-6050 IMU for direct heading measurement would resolve this.

Furthermore, the current system relies on direct straight-line point-to-point navigation and lacks the ability to detect or avoid obstacles in its path. To resolve this, future iterations will implement a dual-layer obstacle avoidance strategy. Globally, the central command centre will use the overhead camera feed to identify static obstacles and compute collision-free waypoints using the A* search algorithm. Locally, an onboard HC-SR04 ultrasonic sensor will be integrated to detect dynamic obstacles, allowing the ESP32 to autonomously pause and execute bypass manoeuvres before resuming its designated path. The current system also tracks a single fire source; multi-fire prioritisation and sequential suppression are planned for future iterations.

Additional planned improvements include: automatic pixel-per-cm calibration from boundary ArUco markers, an onboard ESP32-CAM module for fallback localisation, $LiFePO_4$ battery replacement for improved cycle life, a larger water reservoir for extended suppression, and a cloud-connected logging dashboard for deployed units.

X. Project Links

All project materials are publicly available as required by submission guidelines:

- **GitHub Repository:** https://github.com/MrinmoyPranto008/Image-Processing-Based-Autonomous-Fire-Detection-and-Extinguish
- **Overleaf Source:** https://www.overleaf.com/read/kqcvhsmxnbzr#5921b3
- **Demonstration Video:** https://youtube.com/shorts/FQvPirdNfo0?si=kD56mlDJcpYuSfet

XI. Conclusion

This paper presented a complete autonomous fire detection and suppression system built for under USD 48. The distributed architecture — YOLOv8 inference on a laptop combined with encoder closed-loop burst movement on an ESP32 — achieves reliable fire detection, accurate navigation, and dependable suppression without requiring expensive onboard computing. Experimental trials confirmed 80% full-system success, with the single failure attributable to a transient network issue rather than algorithmic error. The use of dynamic ArUco-based pixel calibration, fire-base tracking to eliminate flame-flicker jitter, FreeRTOS-yielding burst loops to prevent UDP starvation, and the smart-flush stop-command protocol collectively address the practical failure modes encountered during development. The system demonstrates that vision-guided autonomous firefighting is achievable at a cost accessible to small factories and public buildings in developing economies.

References

[1] H. Wang, X. Fu, and Z. Yu, "DSS-YOLO: An improved lightweight real-time fire detection model based on YOLOv8," *arXiv preprint*, 2025.

[2] S. Ma, W. Li, and G. Zhang, "A lightweight fire detection algorithm based on YOLOv8," *IEEE Access*, 2024.

[3] P. H. C. De Venancio, A. C. Rezende, and T. De Aquino Afonso, "D-Fire: An Image Dataset for Fire and Smoke Detection," GitHub, 2022. Available: https://github.com/gaiasd/DFireDataset

[4] Roboflow, "Fire Detection Dataset," Available: https://roboflow.com

[5] Ultralytics, "YOLOv8: Real-Time Object Detection," 2023. Available: https://docs.ultralytics.com

[6] OpenCV Documentation, "ArUco marker detection," 2024. Available: https://docs.opencv.org/4.x/d5/dae/tutorial_aruco_detection.html

[7] Espressif Systems, *ESP32 Technical Reference Manual*, 2023. Available: https://www.espressif.com